Technical Documentation

Emotion Detection from Video using YOLOv8

Live demo: *https://emotion-detection-yolo.streamlit.app/*
Source code: https://github.com/MitaleeGarg29/Emotion-Detection-YOLO

Project Overview

This project builds a two-stage pipeline that detects human faces in video (webcam or recorded file) and classifies the emotion expressed by each detected face in real time. It was built as a technical assessment task and is also published as a personal portfolio project.

Pipeline in one sentence: video frame → YOLOv8 detects face locations → each face is cropped → a CNN classifies the emotion in each crop → results are drawn back onto the frame and exposed via REST API.

---

1. Task Selection & Scoping

The assessment offered three possible tasks:

1. Translation model fine-tuning and deployment
2. An intelligent travel-planning AI agent
3. Emotion detection from video using YOLOv8

Task 3 was selected based on direct alignment with prior hands-on experience: YOLO-based object detection fine-tuning (Fraunhofer LBF, industrial defect detection pipeline) and edge/embedded deep learning deployment (IEEE TENSYMP 2020 publication, GAN-based image dehazing deployed on NVIDIA Jetson Nano). This task plays to demonstrated strengths in computer vision and applied deep learning rather than newer areas like agentic LLM orchestration (Task 2) or translation-specific NLP (Task 1).

2. Architecture Decision: Two Separate Models, Not One

The brief's opening line says, "Use *YOLOv8 to process video for detecting human facial emotions. This task combines object detection with emotion classification."*

At first glance this could be read as "one YOLO model that directly outputs emotions." However, the **Requirements section explicitly separates this into two sub-tasks**, each with its own named dataset:

Sub-task	**Model**	**Dataset**	**What it learns**
Face detection	YOLOv8 (fine-tuned)	WIDER FACE	"Where is a face in this image?"
Emotion classification	Custom CNN	FER-2013	"What emotion does this cropped face show?"

Two models are the correct interpretation, not just a convenient one, because:

- WIDER FACE contains **only bounding box annotations for faces**, no emotion labels exist in this dataset. It is structurally incapable of teaching a model to recognize emotion.
- FER-2013 contains **only pre-cropped, labeled face images with emotion classes**, no bounding box / localization data. It is structurally incapable of teaching a model to detect *where* a face is in a larger scene.
- The Evaluation Criteria section separately lists "fine-tuning YOLO **and** integrating emotion recognition" as two distinct skills being assessed.

Since neither dataset alone can produce a single "detect + classify emotion" model, using two purpose-built models is the only architecture consistent with the actual data provided. The brief's summary sentence describes the **end-user outcome** (an emotion-aware video pipeline); the requirements describe the **two-model implementation** that achieves it.

Additional engineering rationale for keeping them separate (beyond just matching the brief):

- **Separation of concerns**: detection (localization) and classification (fine-grained visual analysis) are fundamentally different sub-problems. YOLO's architecture is optimized for finding object boundaries across a whole scene; a compact CNN is better suited to picking apart subtle pixel-level differences within a small, already-localized crop (e.g., a slightly raised eyebrow vs. a furrowed one).
- **Modularity**: either model can be upgraded or replaced independently later (e.g., swapping in a better emotion architecture) without retraining the other.

---

3. Dataset Preparation

3.1 Face Detection Dataset: WIDER FACE

- **Source**: Kaggle (`lylmsc/wider-face-for-yolo-training`), pre-converted into YOLO format (image + `.txt` label per image).
- **Why WIDER FACE**: It is a benchmark specifically built around *difficulty*, faces at extreme scales, heavy occlusion, blur, varied pose, and lighting. Training on "easy," clean headshots would produce a detector that breaks under real-world conditions (bad lighting, side angles, partial occlusion). A WIDER FACE forces the model to generalize.
- **Verification performed before trusting the dataset**: inspected raw label file contents to confirm the format actually matches YOLO's expected `class_id x_center y_center width height` (normalized 0–1) convention, rather than assuming based on the dataset's title.
- **Total size**: 12,880 images, 12,880 matching label files (1:1 correspondence verified).
- **Train/validation split**: 80/20 (10,304 train / 2,576 validation), fixed random seed (42) for reproducibility.

3.2 Emotion Classification Dataset , FER-2013

- **Source**:Kaggle (`astraszab/facial-expression-dataset-image-folders-fer2013`), organized into folders by class.

- **Why FER-2013:** It is the standard benchmark for facial emotion recognition; its small image size (48×48 grayscale) makes it practical to train a lightweight CNN quickly, which is appropriate for a compact, real-time-oriented pipeline.

- **Classes** (numeric folder names in this dataset release, mapped as follows):

Folder	Emotion
0	Angry
1	Disgust
2	Fear
3	Happy
4	Sad
5	Surprise
6	Neutral

- **Class distribution** (train split): Angry 3,995 / Disgust 436 / Fear 4,097 / Happy 7,215 / Sad 4,830 / Surprise 3,171 / Neutral 4,965 , a known, expected imbalance for this dataset (Disgust is always the minority class in FER-2013). Handling of this imbalance (class-weighted, Focal Loss) is documented in Sections 5.4 and 5.3a.

3.3 Data Engineering Challenge: Google Drive vs. Local Compute

Since training was done on Google Colab (free tier, no persistent local disk) with results saved to Google Drive for permanence, an infrastructure problem was hit and solved:

Problem: Copying tens of thousands of small extracted files (raw images + label files) directly to Google Drive is slow and unreliable, Drive's networked filesystem is not

designed for many small-file writes, and a large copy operation silently produced an incomplete/corrupted copy (1,680 of 12,880 images survived a sync).

Solution adopted:

1. Raw dataset zip files are stored in Drive permanently (single large file, fast, reliable transfer).
2. Each working session, the zip is extracted to Colab's local disk (`/content/...`), fast, since local I/O has no network overhead.
3. All data preparation (train/val split) is performed once on local disk, then **re-zipped into a single file,** and that single file is saved back to Drive, avoiding ever writing thousands of small files to Drive directly.
4. Each future session simply unzips this one prepared file locally (~1–2 minutes) rather than repeating the split logic or risking another Drive sync failure.
5. Trained model weight files (small, single files, a few MB) are saved directly to Drive with no issues, since they don't hit the same many-small-files problem.

This is a deliberate infrastructure decision, not an incidental workaround, it reflects a real constraint of free-tier cloud notebook environments and is worth documenting as a problem-solving example.

---

4. Face Detector: YOLOv8 Fine-Tuning

4.1 Base Model Choice

`yolov8n.pt` (nano), the smallest YOLOv8 checkpoint, pretrained on COCO (80 general object classes).

Why nano, not a larger variant? The assessment explicitly requires real-time performance and an FPS benchmark. Larger YOLOv8 variants (s/m/l/x) may offer marginally better accuracy but at meaningfully higher inference latency, a direct speed/accuracy tradeoff. Given the end-to-end pipeline must run detection *and* classification per frame, minimizing per-stage latency was prioritized.

4.2 Why "Fine-Tuning" and Not Training From Scratch

`yolov8n.pt` retains its COCO-pretrained weights for all general visual feature-extraction layers (edges, textures, shapes, object boundary concepts). Only the

final detection head is reconfigured (from 80 classes down to 1: "face"). Confirmed in training logs: *"Transferred 319/355 items from pretrained weights"*, the vast majority of the network is reused, with only the class-specific head trained fresh. This is standard, textbook fine-tuning and requires dramatically less data/time than training from random weights.

4.3 Training Configuration & Rationale

Parameter	**Value**	**Reasoning**
`epochs`	30	Sufficient for fine-tuning from a pretrained base; training-from-scratch would need far more
`imgsz`	640	YOLOv8 standard input resolution, balances accuracy vs. speed
`batch`	16	Fits comfortably in T4 GPU memory (14.9GB) without OOM errors
`patience`	10	Stops training early if validation metrics plateau for 10 epochs, avoiding wasted compute
`hsv_v` (brightness aug.)	0.4	Directly targets the brief's named challenge of **lighting** variation
`fliplr` (horizontal flip)	0.5	Faces are naturally symmetric under mirroring; doubles effective data variety at no extra collection cost
`mosaic`	1.0	Combines 4 training images into one composite, forcing the model to detect small/partially-cropped faces , directly builds robustness to

Parameter	Value	Reasoning
		the brief's named challenge of **occlusion**
`degrees` (rotation aug.)	5.0	Kept subtle , real-world camera footage has mildly tilted heads, not extreme rotation, so aggressive rotation augmentation would not reflect deployment conditions
`close_mosaic`	10	Disables mosaic augmentation for the final 10 epochs so the model "settles" on realistic, non-composited images before finishing , a known best practice, since real inference input is never mosaiced

4.4 Data Quality Handling

During training data scanning, YOLO's built-in validation automatically identified and handled minor annotation noise in the source dataset:

- 1 image with out-of-bounds/non-normalized coordinates, automatically excluded from training.
- 3 images with duplicate label lines for the same faces, automatically de-duplicated.

This is expected for any large, human-annotated dataset and reflects standard data-quality handling rather than a defect introduced by this project's pipeline.

4.5 Training Results

Final validation metrics (2,576 held-out validation images, never seen during training):

Metric	Value
Precision	83.50%

Metric	Value
Recall	54.92%
F1-score	66.26%
mAP50	61.11%
mAP50-95	31.51%
Inference speed	~5.2ms/image (~190 FPS theoretical, single-image, T4 GPU)

Interpretation: Precision is strong when the model reports a face; it is correct 83.5% of the time. Recall is the weaker figure, at 54.9%, meaning roughly 45% of actual faces in the validation set are missed. This gap is consistent with the deliberate tradeoff made in model selection (Section 4.1): the smallest, fastest YOLOv8 variant was chosen specifically to prioritize real-time inference speed for the video pipeline, at a known cost to recall on WIDER FACE's hardest examples (very small faces, heavy occlusion, extreme angles). This tradeoff was validated informally on a real, unposed group photograph (six faces, varied angles, one face partially turned with sunglasses) during manual testing, where the detector correctly identified all six faces at 0.74–0.82 confidence, a result better than the aggregate validation recall would suggest, consistent with the fact that WIDER FACE's difficulty is concentrated in extreme cases (crowd scenes, very small faces) not typically present in a standard front-facing video use case.

---

5. Emotion Classifier: CNN on FER-2013

5.1 Why a Custom CNN, Not a Fine-Tuned Pretrained Model

FER-2013 images are small (48×48 pixels) and grayscale, very different from the large, colorful, real-world photos (typically 224×224 RGB) that popular pretrained models like ResNet or VGG were trained on. Fine-tuning a large pretrained network on such small, low-resolution grayscale images adds unnecessary size and complexity without a clear accuracy benefit. The standard, well-established approach for FER-2013 specifically (used across most published research on this exact dataset) is a **compact, purpose-built CNN** sized appropriately for this input resolution.

5.2 Base Architecture

Three convolutional blocks with increasing depth (32 → 64 → 128 → 256 channels), each followed by batch normalization and max-pooling, then a global average pooling layer and two dense layers ending in a 7-way output (one score per emotion class).

In simple terms, what each piece does and why it's there:

Component	**Plain-language explanation**
Convolutional layers (conv1–conv4)	Each layer looks for patterns, starting simple (edges, light/shadow boundaries) and building up to complex ones (an eyebrow shape, a curved mouth), earlier layers see simple shapes, and later layers combine those into whole facial features.
Increasing channel depth (32→64→128→256)	Early on, only a few "pattern detectors" are needed for simple shapes; deeper in the network, many more are needed to represent the much larger number of possible complex facial patterns.
Batch normalization	Keeps the numbers flowing through the network stable and consistent while training, which makes training faster and more reliable.
Max-pooling	Shrinks the image progressively (48→24→12→6 pixels) while keeping only the strongest detected signals, reduces computation and stops the model from fixating on exact pixel positions rather than general patterns.
Dropout (0.4)	Randomly switches off 40% of neurons during each training step, forcing the network not to over-rely on any single pathway, helps prevent memorizing the training images instead of learning generalizable patterns. Particularly useful here since FER-2013 is a

Component	Plain-language explanation
	known noisy, sometimes ambiguously labeled dataset.
Global average pooling (instead of flattening)	Condenses each learned feature map down to a single representative number, rather than keeping every pixel position. This means far fewer parameters in the final dense layers, which lowers the risk of overfitting on a modestly sized, imperfect dataset.

5.3 Innovation Added: Squeeze-and-Excitation (SE) Attention Blocks

What it is, in simple terms: A normal CNN treats every learned feature/pattern as equally important, all the time. An SE block adds a small mechanism that lets the network **dynamically decide which patterns matter most for the specific image currently in front of it and** turns up or down its attention to each one accordingly, a lightweight, learned form of "focus."

Why this was added, in plain terms: Several emotions in FER-2013 look visually very similar and are commonly confused, for example, *fear* and *surprise* both involve wide eyes and an open mouth; the real difference is often subtle (eyebrow shape and mouth tension). A standard CNN has no way to say, "Pay extra attention to the eyebrow-shaped pattern right now", every pattern gets equal weight regardless of what's actually useful for the image at hand. An SE block gives the network exactly that ability: to emphasize whichever specific learned pattern is most useful for telling apart a hard, easily confused pair of emotions on a per-image basis.

Why this is a genuine, defensible addition (not complexity for its own sake):

- It is based on a well-established, peer-reviewed technique (Squeeze-and-Excitation Networks, Hu et al., 2018), not an invented or untested idea.
- It adds very few extra parameters (the internal bottleneck in the SE block keeps it lightweight), so it does not meaningfully slow down inference, important since the final pipeline has a real-time / FPS requirement.
- It directly targets a real, nameable weakness of this specific dataset (visually similar, easily-confused emotion classes) rather than being added generically.

Where it's placed: after the second and fourth convolutional blocks , early enough to refine mid-level features (facial parts), but after enough layers have already built up meaningful patterns for the attention mechanism to have something useful to weigh.

5.4 Class Imbalance Handling

FER-2013's training split is imbalanced; the *Disgust* class has only 436 images versus *Happy*'s 7,215 (a ~16.5x difference). Left unaddressed, a model can achieve deceptively high overall accuracy simply by rarely predicting rare classes at all, since doing so barely affects the aggregate accuracy number while completely failing on that class.

Approach taken: Class weights are computed inversely proportional to each class's frequency and passed into the training loss function. This means mistakes on rare classes (like Disgust) are penalized more heavily than mistakes on common classes (like Happy), forcing the model to actually learn to recognize rare emotions rather than ignoring them.

The formula used:

weight[class] = total_images / (num_classes × count[class])

Why this specific formula, explained simply: the idea is that a class's weight should move in the *opposite* direction of how often it appears. A class with very few images (like Disgust, at 436) gets divided by a small number, producing a large weight. A class with many images (like Happy, at 7,215) gets divided by a large number, producing a small weight. Multiplying by `num_classes` in the denominator simply keeps all the weights centered around a value of roughly 1.0 on average, rather than producing arbitrarily large or tiny numbers , so no single class's weight dominates the loss calculation in an unbalanced way, it just nudges rare classes to matter proportionally more and common classes to matter proportionally less than they would by default. This is a standard, widely used formula for handling class imbalance in classification tasks, not a custom invention for this project.

5.5 Data Augmentation

Applied to the training set only (not validation/test, since those must reflect real, unmodified data for honest evaluation):

- **Random horizontal flip:** Faces are naturally symmetric under mirroring, so this effectively doubles training variety at no extra data-collection cost.
- **Random small rotation (±10°)** , mimicking natural head-tilt variation seen in real footage.
- **Brightness/contrast jitter directly** targets the brief's named challenge of **lighting** robustness, since a deployed pipeline will face varied real-world lighting conditions.

5.6 Training Configuration & Results

Training setup:

- Optimizer: AdamW (lr=0.001, weight_decay=1e-4)
- Loss function: Weighted Focal Loss (see Section 5.3a below)
- Scheduler: ReduceLROnPlateau (halves learning rate if validation loss plateaus for 3 epochs)
- Max epochs: 40, with early stopping after 7 epochs of no validation-loss improvement
- Checkpointing: best model (lowest validation loss) saved to Drive after every improving epoch, with full resume support (epoch number, optimizer state, and validation metrics all saved alongside model weights), which is critical since training ran on CPU across multiple sessions

Final results (test set, 3,589 held-out images, never seen during training or validation):

Overall:

Metric	**Value**
Test accuracy	62.78%
Macro precision	58.00%
Macro recall	63.95%
Macro F1-score	59.66%
Weighted precision	63.74%
Weighted recall	62.78%

Metric	Value
Weighted F1-score	62.93%

Per-class breakdown:

Emotion	Precision	Recall	F1-score	Support (test images)
Angry	0.580	0.593	0.586	491
Disgust	0.341	0.764	0.472	55
Fear	0.438	0.384	0.409	528
Happy	0.901	0.779	0.836	879
Sad	0.482	0.483	0.483	594
Surprise	0.716	0.844	0.775	416
Neutral	0.602	0.629	0.615	626

Interpretation, including honest discussion of where the model struggles:

- **Happy** is the strongest class by a clear margin (F1 = 0.836) , consistent with it being both the largest class in training data and, more fundamentally, the emotion with the most visually distinctive, unambiguous facial signal (a smile).
- **Disgust** shows the clearest effect of the class-weighting and focal loss strategy (Sections 5.3a, 5.4): despite having only 55 test images (by far the smallest class), recall reaches 76.4%; the model is not ignoring this rare class, which is exactly the outcome the weighting was designed to produce. However, precision on "Disgust" is the lowest of any class (0.341), meaning the model over-predicts "Disgust" on images that are not actually Disgust, in exchange for rarely missing true Disgust cases. This precision/recall tradeoff on the rarest class is a direct, visible consequence of deliberately up-weighting it and is worth stating plainly rather than only reporting the favorable recall number.
- **Fear** is the weakest-performing class overall (F1 = 0.409). This is consistent with FER-2013's well-documented difficulty distinguishing fear from visually similar expressions (surprise and, to a lesser extent, sadness), a known, published limitation of this dataset, not specific to this implementation.

- The gap between **macro F1 (0.597)** and **weighted F1 (0.629)** reflects the class imbalance directly: weighted F1 is pulled upward by strong performance on the large Happy class, while macro F1 (which treats every class equally regardless of size) more honestly reflects that several minority/harder classes (Disgust, Fear, Sad) still underperform.

Honest overall assessment: a 62.8% test accuracy across 7 emotion classes is a reasonable, defensible result for a lightweight, from-scratch CNN trained on FER-2013, a dataset widely acknowledged in published research to be noisy and, in a meaningful fraction of cases, ambiguous even for human annotators. This result is presented as-is rather than adjusted or cherry-picked, consistent with the project's broader commitment to not overstating results.

5.3a Loss Function Innovation: Weighted Focal Loss

In addition to class weighting (Section 5.4), the loss function itself was upgraded from standard weighted cross-entropy to **weighted focal loss,** adapted from Lin et al., 2017 ("Focal Loss for Dense Object Detection," the RetinaNet paper).

In simple terms: standard cross-entropy loss treats every mistake the same, regardless of whether the model was "barely wrong" or "completely lost" on that example. Focal Loss adds a modulating term, `(1 - p_t)^gamma`, which **automatically shrinks the loss contribution from examples the model already classifies confidently and correctly while keeping full loss weight on hard, confusing examples**. Combined with class weighting, this means rare classes (via class weights) *and* visually-confusing examples (via the focal term) both get proportionally more of the model's learning attention , two distinct problems this dataset genuinely has, addressed by two complementary mechanisms in the same loss function.

Why this matters specifically for FER-2013: Several emotion pairs are visually similar and frequently confused (e.g., fear vs. surprise, angry vs. disgust) even by human annotators. Plain cross-entropy spends a large share of training "effort" on the easy majority of images (clear, obvious happy/neutral faces) that the model already classifies correctly early on, which dilutes the learning signal available for the harder, more ambiguous cases. Focal Loss redirects that effort toward exactly the examples that need it.

Why this is a legitimate, citable choice: it is a well-established, peer-reviewed technique, originally designed to address class imbalance in object detection, applied here for the analogous imbalance/difficulty problem in emotion classification.

5.7 Full Model Architecture

Total parameters: 440,367 , deliberately lightweight (for comparison, ResNet-18 has ~11 million parameters), keeping inference fast enough not to bottleneck the real-time pipeline.

5.8 Loading the Trained Model (for inference / pipeline use)

Since PyTorch checkpoints store only the trained weight *values*, not the architecture code itself, the model class must be redefined (Section 5.7) before loading the saved weights into it:

```
emotion_model = EmotionCNN(num_classes=7).to(device)

checkpoint = torch.load(

    f'{PROJECT_ROOT}/models/emotion_classifier/best_emotion_model.pt',

    map_location=device

)

emotion_model.load_state_dict(checkpoint['model_state_dict'])

emotion_model.eval()     # switches to inference mode: disables dropout, freezes batch-norm statistics

emotion_names = {0: 'Angry', 1: 'Disgust', 2: 'Fear', 3: 'Happy', 4: 'Sad', 5: 'Surprise', 6: 'Neutral'}
```

`.eval()` is important here; it is not optional boilerplate. In training mode, dropout randomly disables neurons, and batch-norm uses per-batch statistics; in evaluation/inference mode, the full network is used deterministically with its learned, fixed statistics, necessary for consistent, reproducible predictions in the actual pipeline.

---

6. Pipeline Integration

Design: video frame → YOLOv8 face detection → crop each detected face region → preprocess crop (resize to 48×48, grayscale) to match FER-2013's input format → CNN emotion classification → annotate original frame with bounding box + emotion label → output (displayed live or returned via API). This repeats independently for every frame in the video stream.

6.1 Confidence-Based Uncertainty Handling

Rather than always displaying the CNN's top predicted class regardless of how confident that prediction actually is, the pipeline applies a confidence threshold (45%): if the classifier's top prediction falls below this threshold, the frame is labeled **"Uncertain"** rather than committing to a specific emotion the model isn't confident about.

Why this matters: given the CNN's known per-class limitations (Section 5.6 above , fear and sadness in particular are frequently confused with visually similar classes), silently displaying a low-confidence guess as if it were certain would misrepresent the system's actual reliability. Explicitly surfacing uncertainty is a more honest and more useful behavior for a real video pipeline than forcing a guess on every single face in every single frame.

6.2 Temporal Smoothing

Running detection and classification independently on every frame can produce visible "flicker" , a face detected in one frame but missed in the next due to a small lighting or angle change, or an emotion label rapidly switching between two similar classes frame-to-frame even though the person's actual expression hasn't changed. To address this:

- Detected faces are tracked across consecutive frames (matched by spatial overlap between frames, i.e. IoU-based tracking) so the same physical face is recognized as the same tracked entity over time, not re-detected from scratch every frame.
- Emotion predictions for a given tracked face are smoothed over a short recent window of frames, rather than reacting instantly to a single frame's classification. This reduces label flicker and produces a more stable, realistic-looking output video.

This directly addresses the brief's named challenge of handling video-specific behavior (as distinct from static-image detection), and reflects a deliberate engineering addition beyond the minimum "detect + classify per frame" requirement.

6.3 Batch Processing & Reporting

Beyond single-video processing, the pipeline supports batch processing of multiple videos in one run, producing per-video and aggregate outputs: annotated output videos, CSV logs of every frame's detections and confidence scores, and summary charts (emotion distribution over time, per-video comparison), supporting the kind of analysis a real deployment would need beyond a single demo clip.

---

7. Evaluation Methodology

Why image-based test sets are used for accuracy metrics (precision/recall/F1), not video: Standard accuracy metrics require ground-truth labels to compare against. No accessible public dataset provides per-frame emotion labels for video. Therefore:

- **Face detector accuracy** is measured against WIDER FACE's own held-out validation split (2,576 images, never seen during training).
- **Emotion classifier accuracy** is measured against FER-2013's dedicated test split (separate from its training data).

Why video is still used, separately: Real-time performance (FPS benchmark) and the live/recorded demo required by the brief are tested using self-recorded video footage , capturing a range of expressions, lighting, and angles to demonstrate the pipeline's practical, end-to-end behavior. This is a speed/functionality test, distinct from the statistical accuracy evaluation above.

Final measured results are reported in Sections 4.5 (face detector) and 5.6 (emotion classifier) above.

---

8. Deployment

8.1 REST API

A FastAPI service exposes the pipeline as a REST API, accepting an uploaded video file and returning per-frame detections (bounding boxes, predicted emotion, confidence score) as structured JSON, satisfying the brief's requirement for "a REST API to process video streams or files, returning emotions with bounding boxes for each frame." The API includes a `/health` endpoint and a `/process-video` endpoint, and was manually tested via file upload.

8.2 Web Interface

A Streamlit-based web interface allows a user to upload a video directly in the browser, view the annotated output with bounding boxes and emotion labels, and download the processed result. This satisfies the brief's requirement for "a simple web app to display live results."

8.3 Cloud Deployment: Platform Choice and Honest Tradeoffs

What was deployed: the web interface is deployed on **Streamlit Community Cloud**, a free hosting service for Streamlit applications, giving the project a permanent, publicly accessible URL independent of any local machine or development notebook.

On the brief's request for a "GPU-enabled cloud platform": this was evaluated directly and the outcome is documented honestly here rather than glossed over:

- **Hugging Face Spaces** was the first platform attempted. As of this project's development, Hugging Face's free tier no longer permits hosting compute-backed Spaces (Gradio or Docker SDKs, which is what a Streamlit or Gradio app requires to run) without a paid PRO subscription , a policy change from what was previously a straightforwardly free option. The one remaining free GPU path, **ZeroGPU** (shared, time-sliced NVIDIA A100 access), requires both the Gradio SDK specifically (not Streamlit) and an account meeting a minimum age/verification threshold not met at the time of this project.
- **True persistent GPU cloud hosting** (e.g., a continuously-running AWS/GCP GPU instance) has a genuine ongoing cost and is not realistic to maintain for a portfolio/assessment project without incurring recurring charges.

- **Decision made:** deploy on CPU-based infrastructure (Streamlit Community Cloud's free tier), a deliberate choice justified by the project's own model design; both the face detector (YOLOv8n, the smallest available variant) and the emotion classifier (a 440K-parameter custom CNN) were specifically built to be lightweight, precisely so that real-time-oriented inference would not be GPU-dependent. CPU inference speed was verified to be adequate for the deployed use case (uploaded video processing, not live multi-stream production traffic).

This is presented as a documented engineering tradeoff, not an oversight: given a choice between (a) an idle/non-functional GPU deployment blocked by a platform's paid-tier requirement or (b) a fully working, freely accessible CPU deployment enabled by deliberately lightweight model design, option (b) was chosen as the more genuinely useful, honestly described outcome , consistent with this project's broader principle of not overstating what was built.

8.4 Known Limitations of the Free-Tier Deployment

- Streamlit Community Cloud's free tier provides approximately 1GB RAM and may put the app to sleep after a period of inactivity, resulting in a brief (30–60 second) cold-start delay on the next visit , standard, expected free-tier behavior, not an application defect.
- CPU inference is slower than GPU inference in absolute terms; for the deployed use case (processing an uploaded short video, not sustained live-stream production traffic), this was verified to be acceptable.

---

9. Honest Scope Notes

In the interest of accurate, defensible documentation (rather than overstating what was built):

- **AffectNet** (listed as an optional dataset alongside FER-2013 in the brief) was not incorporated due to its access-request requirement and significantly larger scale relative to the project timeline. FER-2013 was used as the sole training dataset for emotion classification and is sufficient on its own per the brief's phrasing ("datasets like FER-2013, AffectNet").
- **GPU-enabled cloud deployment**, as requested in the brief, was not achieved in the strictest literal sense. The reasoning, alternatives evaluated, and the

deliberate CPU-based decisions are documented in full in Section 8.3. This is disclosed directly rather than described ambiguously.

- **Emotion classifier test accuracy (62.78%)** is reported as is. FER-2013 is a widely documented noisy dataset (published baselines using comparable lightweight CNN architectures typically report similar or comparable overall accuracy ranges); no attempt was made to selectively report only favorable metrics. The full per-class breakdown, including the classifier's weaker classes (fear, sad), is included in Section 5.6.
- **Face detector recall (54.9%)** reflects a deliberate speed/accuracy tradeoff (smallest YOLOv8 variant, prioritizing real-time inference) rather than a training failure; this tradeoff is stated explicitly in Section 4.5 rather than left for a reader to infer on their own.